NATURAL LANGUAGE PROCESSING

LABORATORY ASSIGNMENT - 04

Country Prediction using Word Embeddings

Student Name	Shravani Kurkute
Roll No.	421129
PRN / Registration No.	22311926
Class / Division	BTechA
Academic Year	2026-2027
Faculty Name	Dr. Kapil N. Vhatkar

Assignment 4: Country Prediction using Word Embeddings

Aim: To understand semantic relationships represented by word embeddings and use vector arithmetic and similarity measures to predict a country from capital-city relationships.

1. Objectives

- Understand the concept and properties of pre-trained word embeddings.
- Load and use a suitable pre-trained word-embedding model.
- Perform vector arithmetic to represent word analogies.
- Use a similarity function to identify the closest candidate word.
- Evaluate the country-prediction system on multiple capital-country examples.

2. Problem Statement

Write a Python program that can predict a country from a capital-city relationship using pre-trained word embeddings. The program should accept three words: Capital 1, Country 1 and Capital 2, and predict Country 2. For example, given Athens, Greece and Baghdad, the system should predict Iraq.

3. Example

Input 1	Input 2	Input 3	Expected Output
Athens	Greece	Baghdad	Iraq

4. Theory / Background

Word embeddings represent words as dense numerical vectors in a continuous vector space. Words with related meanings or usage patterns tend to have similar vector representations. A common analogy approach uses vector arithmetic to estimate the missing word.

Vector = vector(Country 1) - vector(Capital 1) + vector(Capital 2)

The resulting vector is compared with vocabulary vectors using a similarity measure such as cosine similarity. The highest-ranked suitable candidate is selected as the prediction.

5. Tools, Libraries and Environment

Sr. No.	Tool / Library	Version	Purpose
1	Python	3.12.3	Programming language used for the whole implementation
2	gensim	4.4.0	Loads the pre-trained GloVe vectors (KeyedVectors) and downloads the model
3	NumPy	2.4.4	Vector arithmetic, normalization and cosine similarity (matrix dot products)
4	SciPy	1.17.1	Numerical dependency of gensim
5	GloVe glove-wiki-gigaword-100	100-d, 400K words	Pre-trained word-embedding model (Wikipedia 2014 + Gigaword 5)
6	csv / json / argparse / random	Python standard library	Reading datasets, saving results, command-line options, fixed-seed test sampling

6. Pre-trained Embedding Model

Model name: **GloVe (Global Vectors) – glove-wiki-gigaword-100**

Source / Provider: **Stanford NLP Group (Pennington, Socher & Manning, 2014); loaded through gensim-data**

Embedding dimension: **100**

Vocabulary size: **400,000 words (lower-cased; trained on Wikipedia 2014 + Gigaword 5, 6 billion tokens)**

File / Model size: **≈ 128 MB compressed download (glove-wiki-gigaword-100.gz)**

Similarity measure: **Cosine similarity**

7. Methodology / Algorithm

1. Load the pre-trained word-embedding model.

2. Read Capital 1, Country 1 and Capital 2 as input.

3. Retrieve the embedding vectors for the three input words.

4. Calculate vector(Country 1) − vector(Capital 1) + vector(Capital 2).

5. Compare the resulting vector with the vocabulary using a similarity measure.

6. Remove the input words and unsuitable candidates from the candidate list.

7. Select the highest-ranked valid candidate as the predicted country.

8. Repeat the experiment for multiple capital-country examples.

9. Calculate prediction accuracy and analyse incorrect predictions.

8. Input / Output Record

Sr. No.	Capital 1	Country 1	Capital 2	Predicted Country
1	Athens	Greece	Baghdad	**Iraq**
2	Kabul	Afghanistan	Beijing	**China**
3	Beirut	Lebanon	Budapest	**Hungary**
4	Vienna	Austria	Madrid	**Spain**
5	Jakarta	Indonesia	Bratislava	**Slovakia**
6	Helsinki	Finland	Vientiane	**Laos**
7	Kathmandu	Nepal	Berlin	**Germany**
8	Beijing	China	Helsinki	**Finland**
9	Tokyo	Japan	Warsaw	**Poland**
10	Sofia	Bulgaria	London	**England**
11	Beijing	China	Valletta	**Malta**
12	Stockholm	Sweden	Bratislava	**Slovakia**

13	Mogadishu	Somalia	Vienna	**Germany**

14	Baku	Azerbaijan	Vientiane	**Laos**
15	Beirut	Lebanon	Athens	**Greece**

9. Similarity / Ranked Candidates

Record the top candidate words and similarity scores for selected examples.

Example	Candidate 1	Score	Candidate 2	Score	Selected Prediction
1	Iraq	0.790	Afghanistan	0.708	**Iraq ✓**
2	China	0.843	Vietnam	0.693	**China ✓**
4	Spain	0.769	Portugal	0.659	**Spain ✓**
6	Laos	0.660	Mozambique	0.533	**Laos ✓**
10	England	0.690	Australia	0.671	**England ✓**
11	Malta	0.681	Liechtenstein	0.557	**Malta ✓**
13	Germany	0.645	Austria	0.637	**Germany ✗**
15	Greece	0.787	Cyprus	0.657	**Greece ✓**

Example numbers refer to the Sr. No. in Section 8. Candidates are ranked by cosine similarity (country-filtered list); ✓ *= prediction matches the actual country,* ✗ *= wrong.*

10. Evaluation

Total test examples: **15**

Correct predictions: **14**

Incorrect predictions: **1**

Accuracy (%): 93. 33 % ($top-5\ accuracy$: 100. 0 %)

Example	Actual Country	Predicted Country	Correct / Incorrect	Observation
1	Iraq	Iraq	**Correct**	Exact match; cosine 0.790, margin 0.082 over 'afghanistan'
2	China	China	**Correct**	Exact match; cosine 0.843, margin 0.150 over 'vietnam'
3	Hungary	Hungary	**Correct**	Exact match; cosine 0.795, margin 0.086 over 'lithuania'
4	Spain	Spain	**Correct**	Exact match; cosine 0.769, margin 0.110 over 'portugal'
5	Slovakia	Slovakia	**Correct**	Exact match; cosine 0.811, margin 0.105 over 'hungary'
6	Laos	Laos	**Correct**	Exact match; cosine 0.660, margin 0.127 over 'mozambique'
7	Germany	Germany	**Correct**	Exact match; cosine 0.816, margin 0.136 over 'poland'

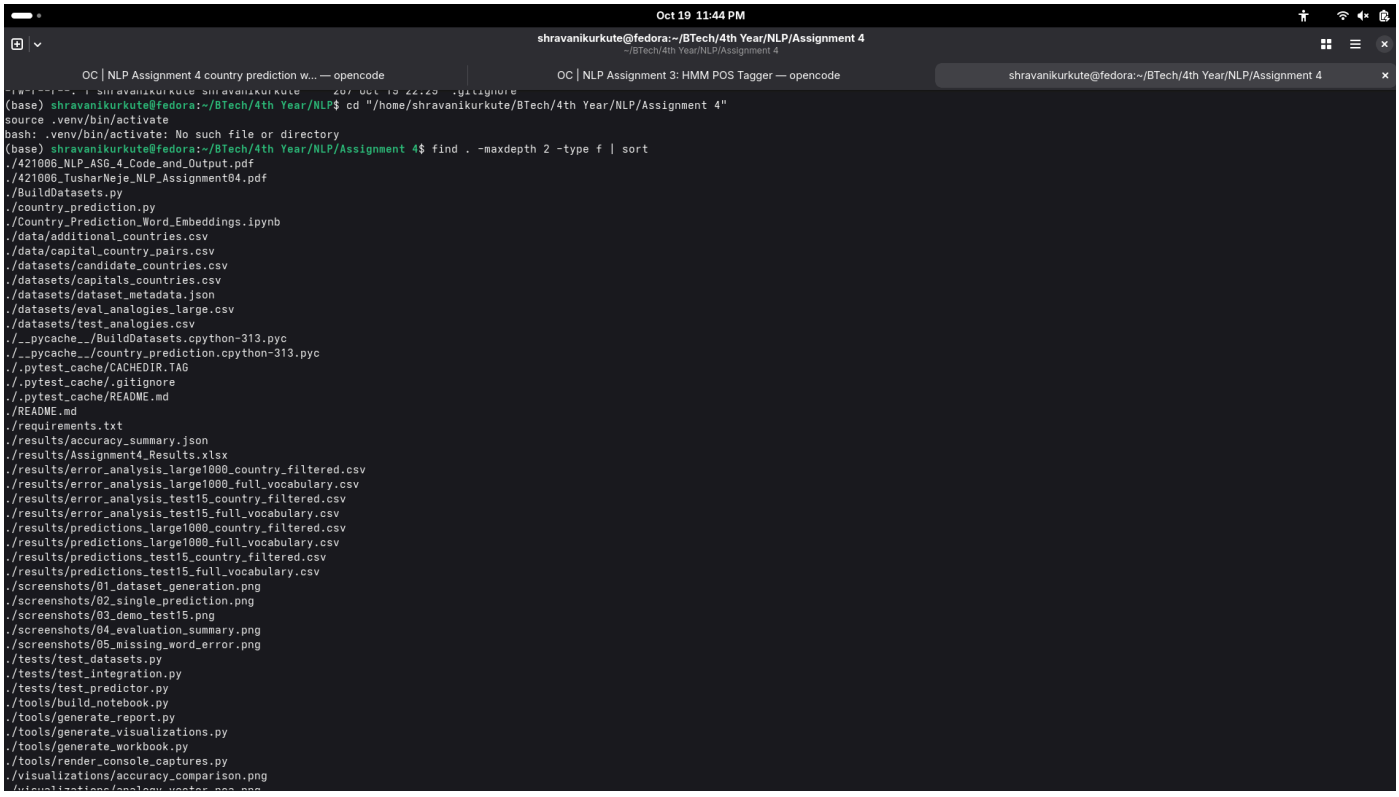
8	Finland	Finland	**Correct**	Exact match; cosine 0.693, margin 0.076 over 'norway'
9	Poland	Poland	**Correct**	Exact match; cosine 0.845, margin 0.134 over 'hungary'
10	England	England	**Correct**	Exact match; cosine 0.690, margin 0.019 over 'australia'

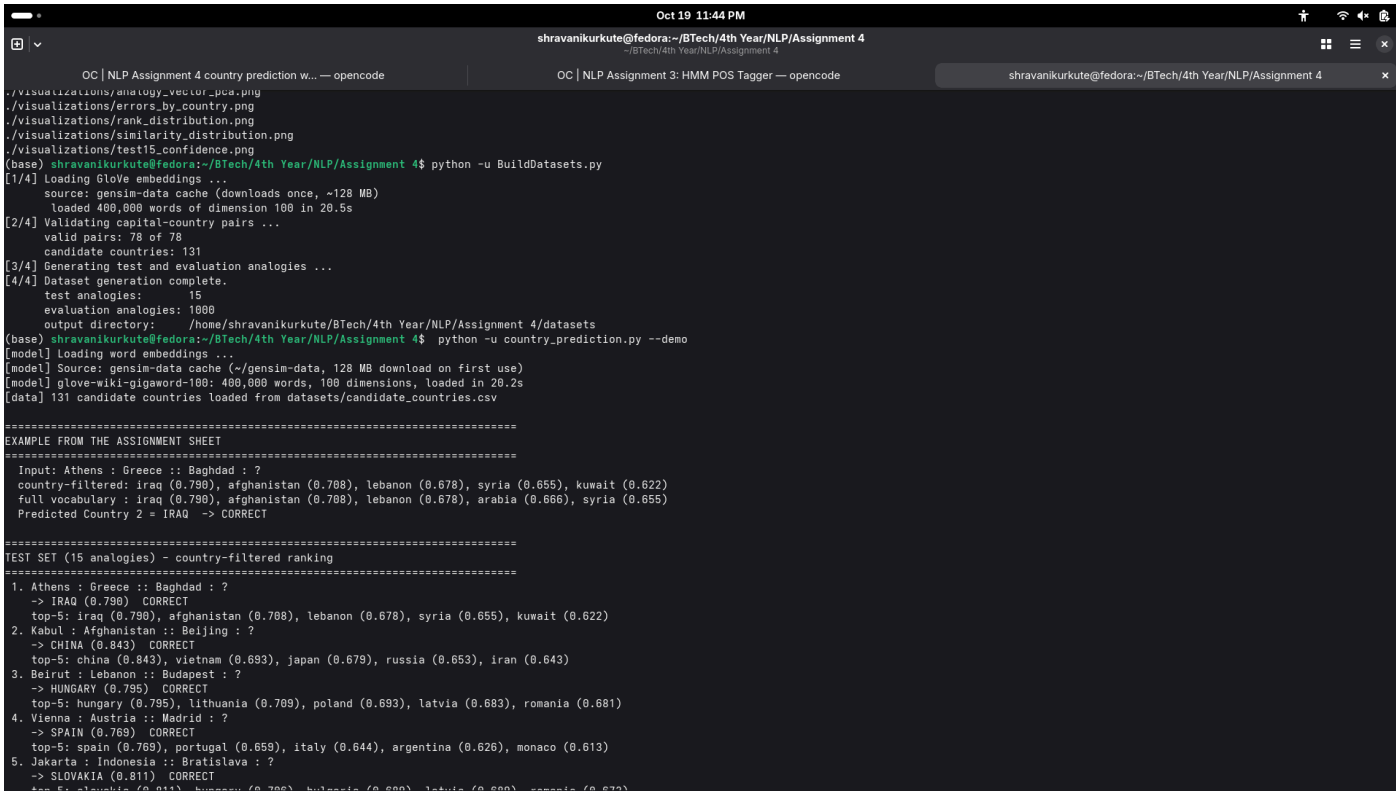
11. Source Code

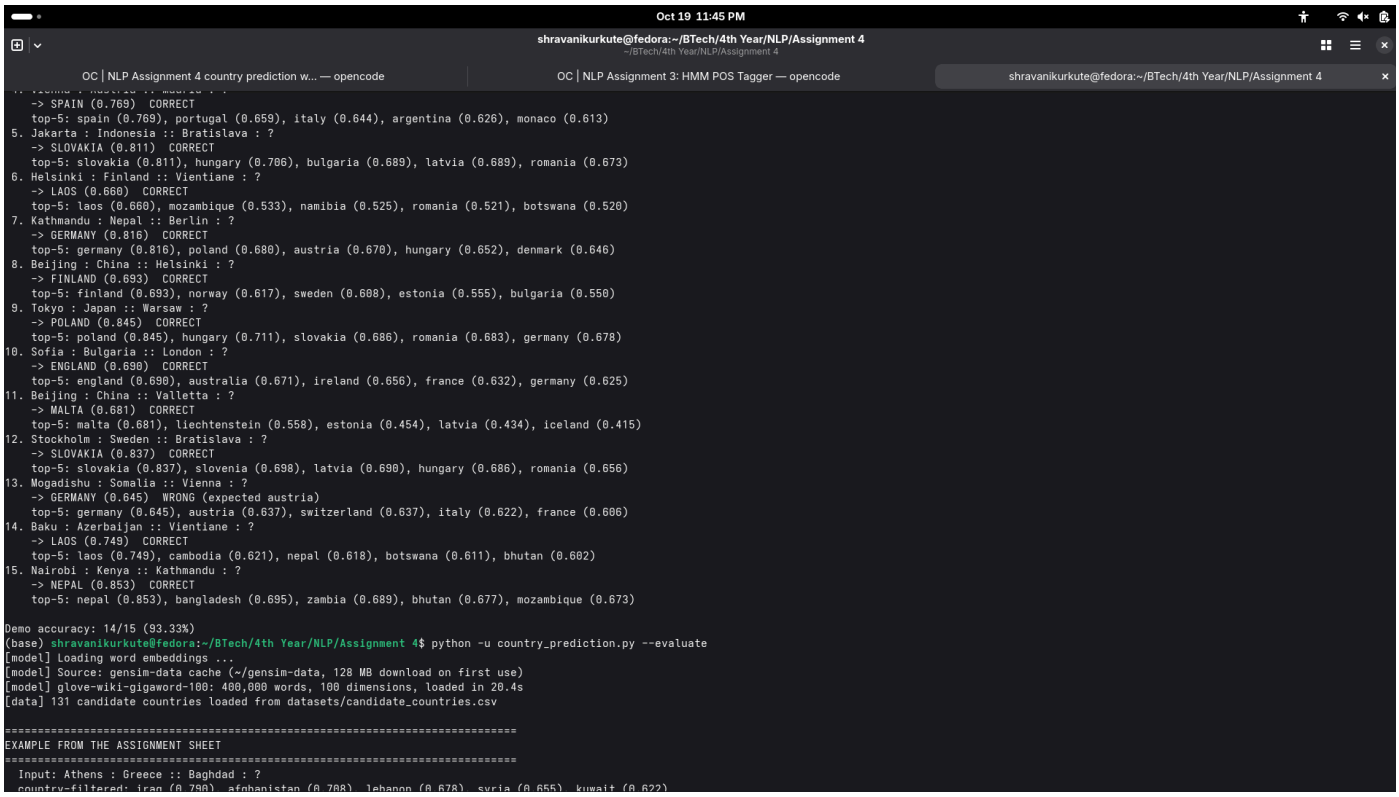
Attach the complete source code separately.

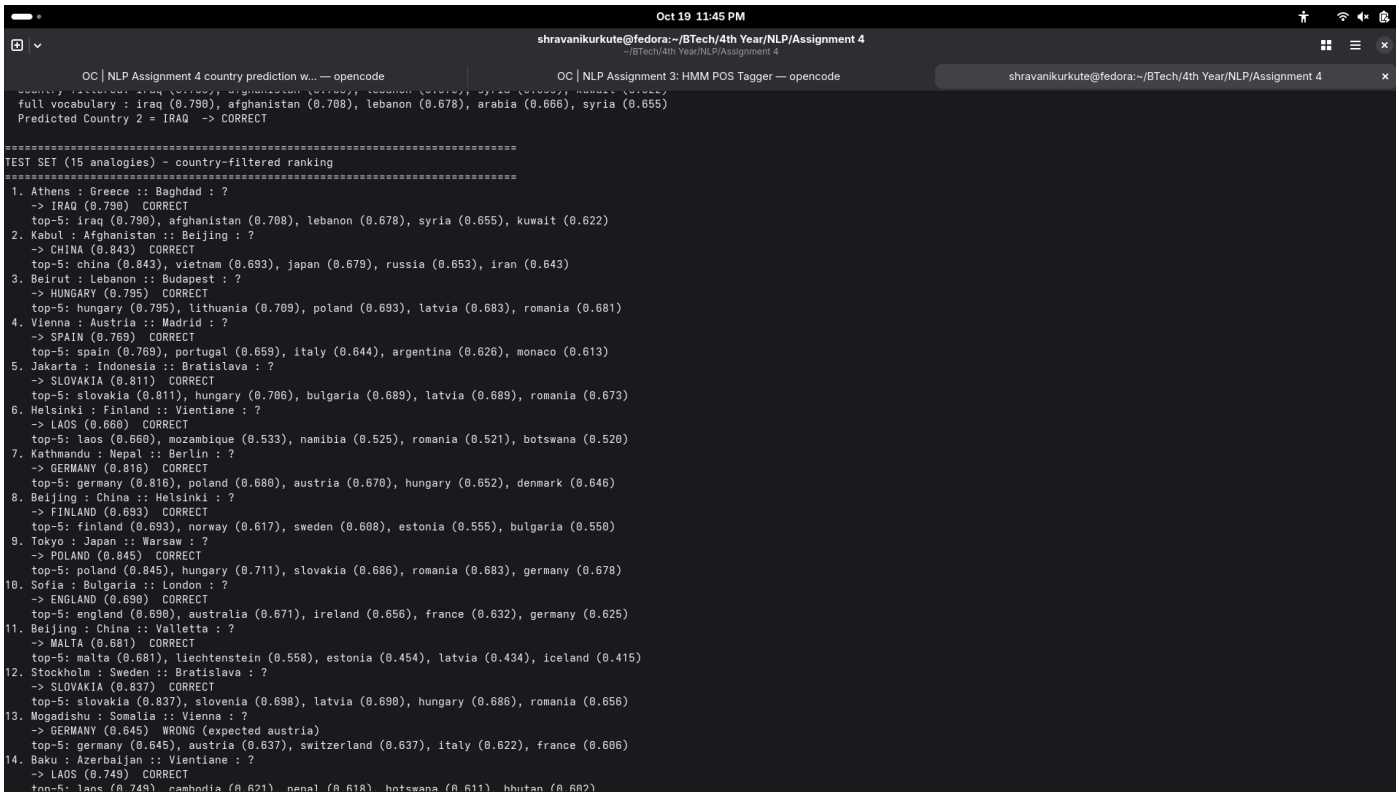
12. Results / Output Screenshots

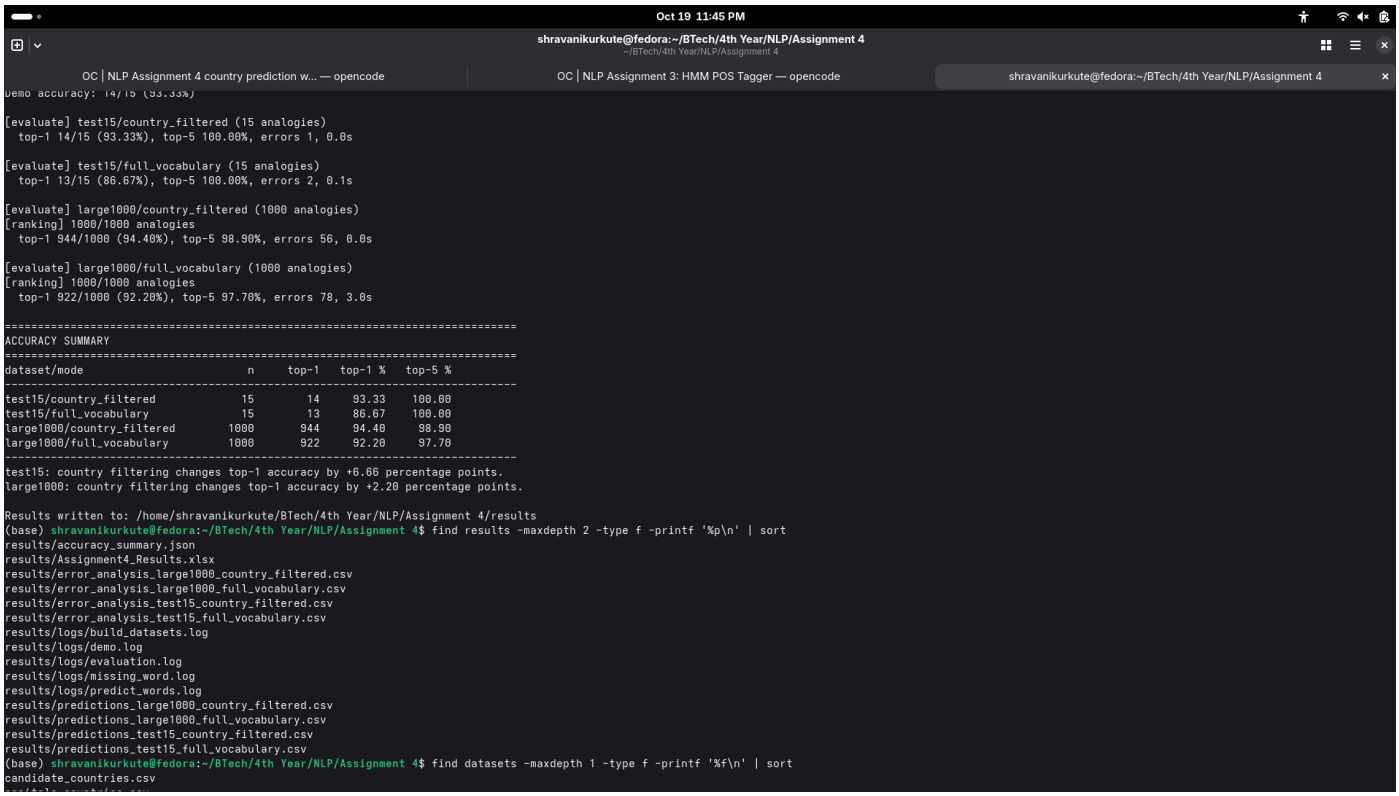
Insert screenshots showing model loading, sample predictions and similarity results.

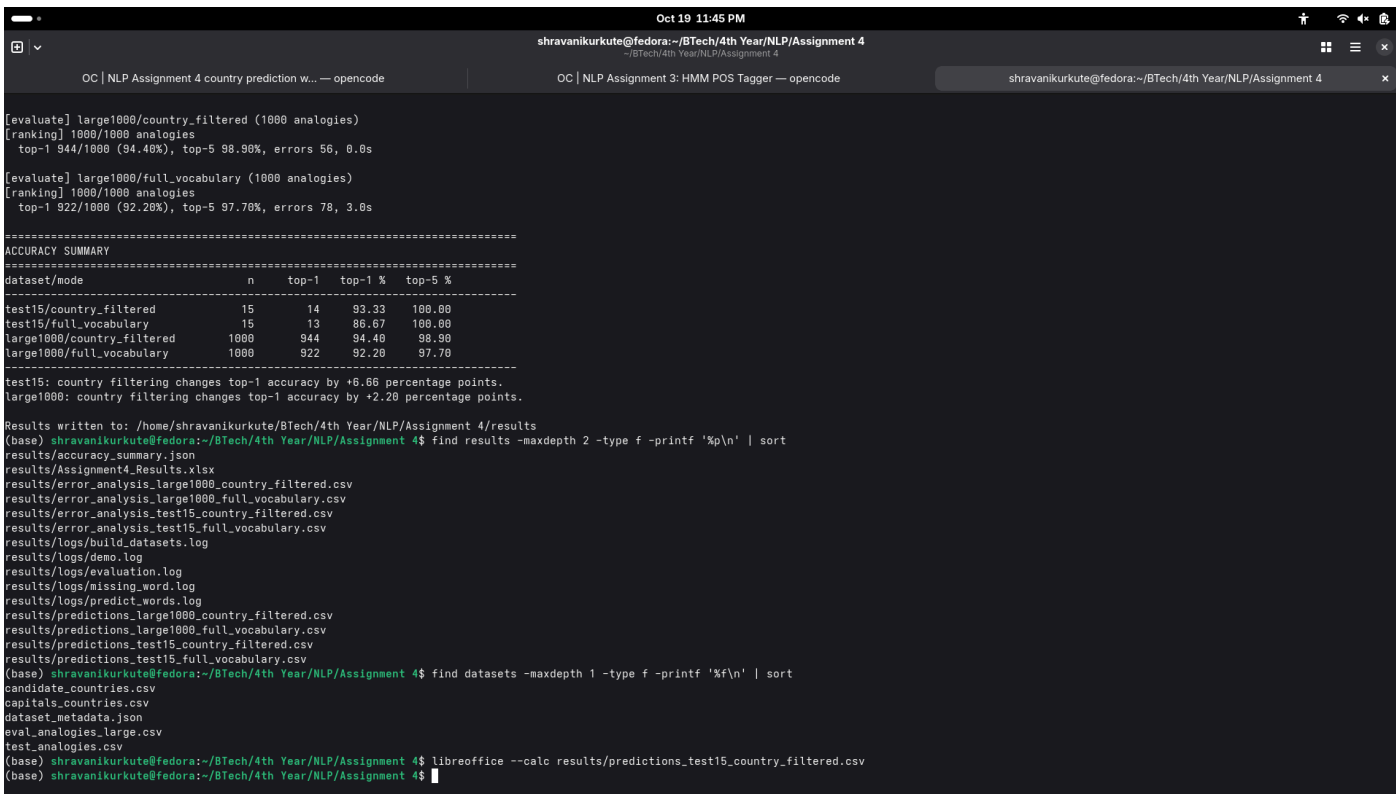


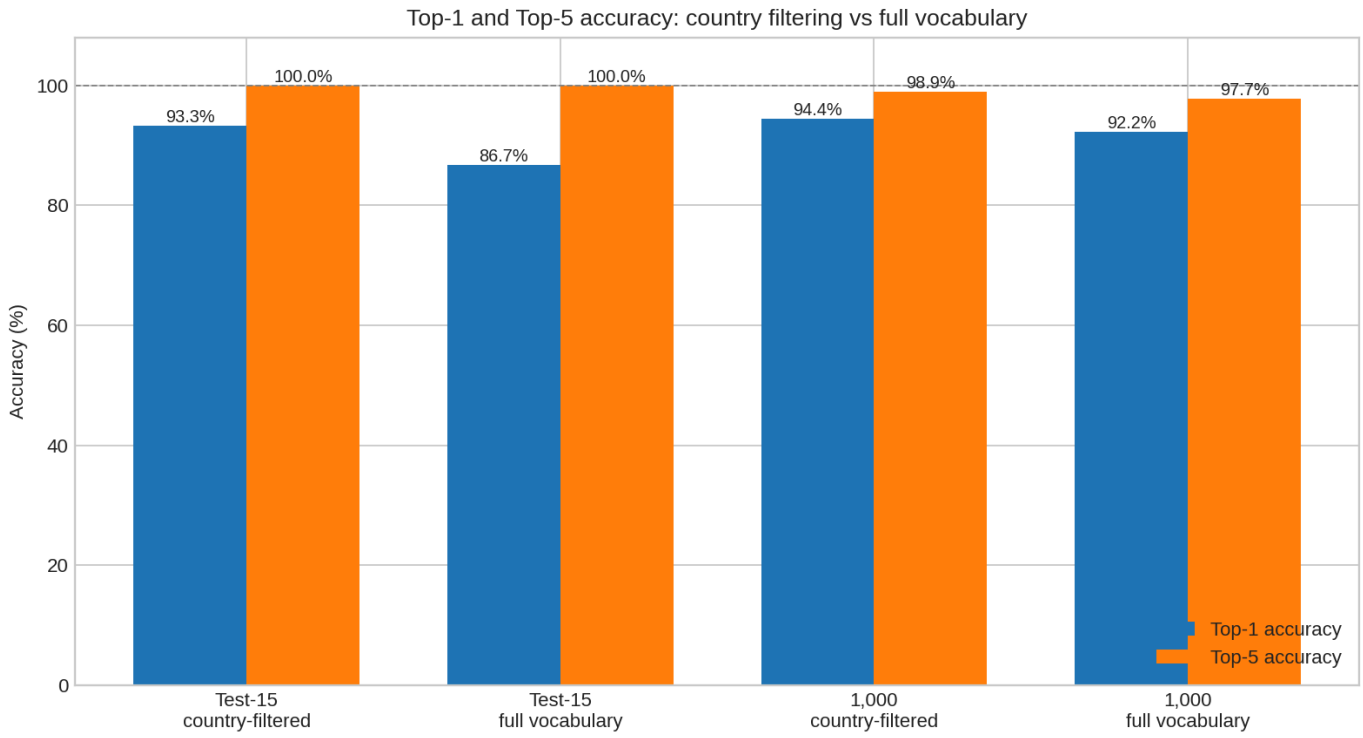


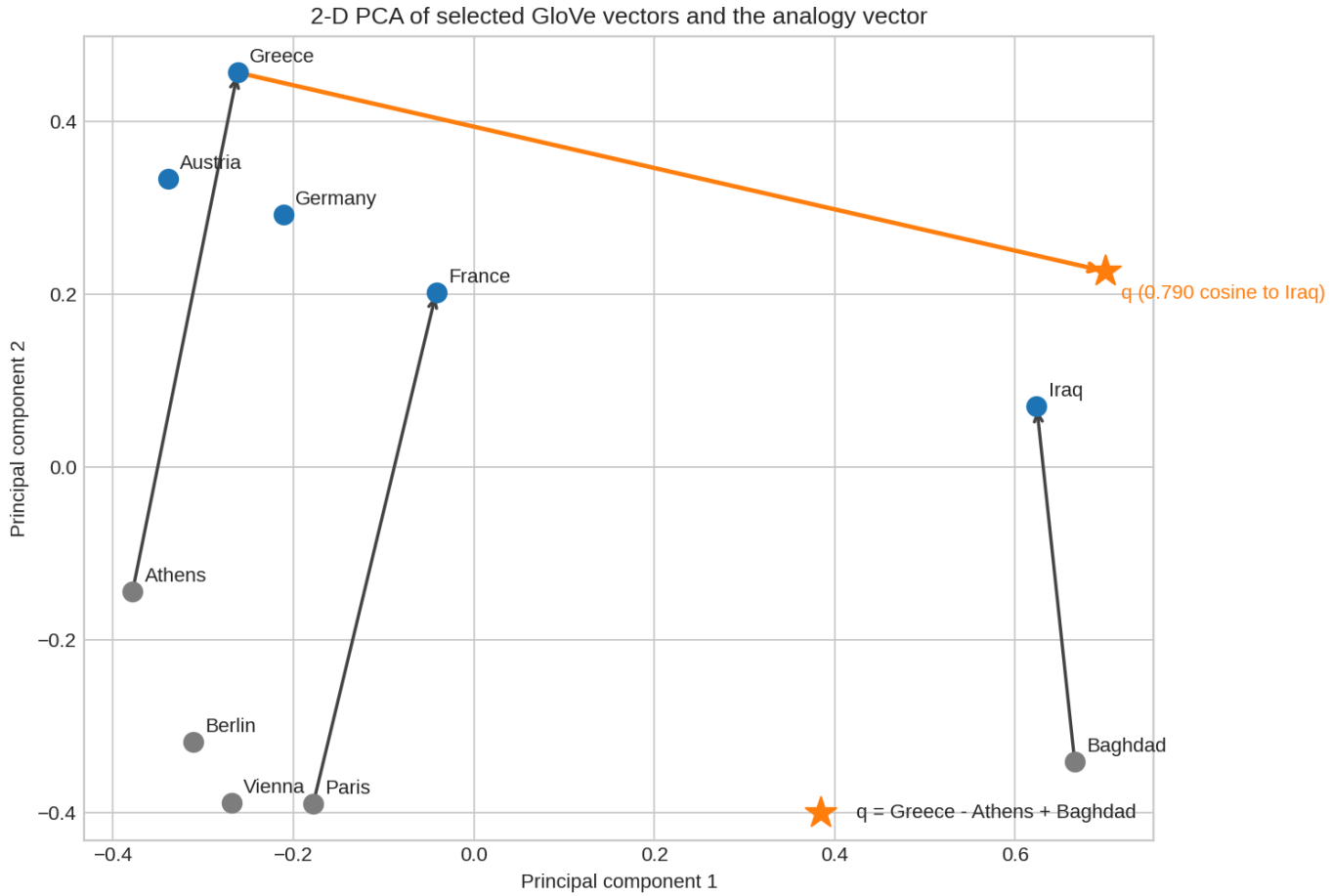

Oct 19 11:45 PM
shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 4
~/BTech/4th Year/NLP/Assignment 4
OC | NLP Assignment 4 country prediction w... — opencode
OC | NLP Assignment 3: HMM POS Tagger — opencode
shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 4
-> SPAIN (0.769) CORRECT
top-5: spain (0.769), portugal (0.659), italy (0.644), argentina (0.626), monaco (0.613)
5. Jakarta : Indonesia :: Bratislava : ?
-> SLOVAKIA (0.811) CORRECT
top-5: slovakia (0.811), hungary (0.706), bulgaria (0.689), latvia (0.689), romania (0.673)
6. Helsinki : Finland :: Vientiane : ?
-> LAOS (0.660) CORRECT
top-5: laos (0.660), mozambique (0.533), namibia (0.525), romania (0.521), botswana (0.520)
7. Kathmandu : Nepal :: Berlin : ?
-> GERMANY (0.816) CORRECT
top-5: germany (0.816), poland (0.680), austria (0.670), hungary (0.652), denmark (0.646)
8. Beijing : China :: Helsinki : ?
-> FINLAND (0.693) CORRECT
top-5: finland (0.693), norway (0.617), sweden (0.608), estonia (0.555), bulgaria (0.550)
9. Tokyo : Japan :: Warsaw : ?
-> POLAND (0.845) CORRECT
top-5: poland (0.845), hungary (0.711), slovakia (0.686), romania (0.683), germany (0.678)
10. Sofia : Bulgaria :: London : ?
-> ENGLAND (0.690) CORRECT
top-5: england (0.690), australia (0.671), ireland (0.656), france (0.632), germany (0.625)
11. Beijing : China :: Valletta : ?
-> MALTA (0.681) CORRECT
top-5: malta (0.681), liechtenstein (0.558), estonia (0.454), latvia (0.434), iceland (0.415)
12. Stockholm : Sweden :: Bratislava : ?
-> SLOVAKIA (0.837) CORRECT
top-5: slovakia (0.837), slovenia (0.698), latvia (0.690), hungary (0.686), romania (0.656)
13. Mogadishu : Somalia :: Vienna : ?
-> GERMANY (0.645) WRONG (expected austria)
top-5: germany (0.645), austria (0.637), switzerland (0.637), italy (0.622), france (0.606)
14. Baku : Azerbaijan :: Vientiane : ?
-> LAOS (0.749) CORRECT
top-5: laos (0.749), cambodia (0.621), nepal (0.618), botswana (0.611), bhutan (0.602)
15. Nairobi : Kenya :: Kathmandu : ?
-> NEPAL (0.853) CORRECT
top-5: nepal (0.853), bangladesh (0.695), zambia (0.689), bhutan (0.677), mozambique (0.673)
Demo accuracy: 14/15 (93.33%)
(base) shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 4$ python -u country_prediction.py --evaluate
[model] Loading word embeddings ...
[model] Source: gensim-data cache (~/gensim-data, 128 MB download on first use)
[model] glove-wiki-gigaword-100: 400,000 words, 100 dimensions, loaded in 20.4s
[data] 131 candidate countries loaded from datasets/candidate_countries.csv
EXAMPLE FROM THE ASSIGNMENT SHEET
Input: Athens : Greece :: Baghdad : ?

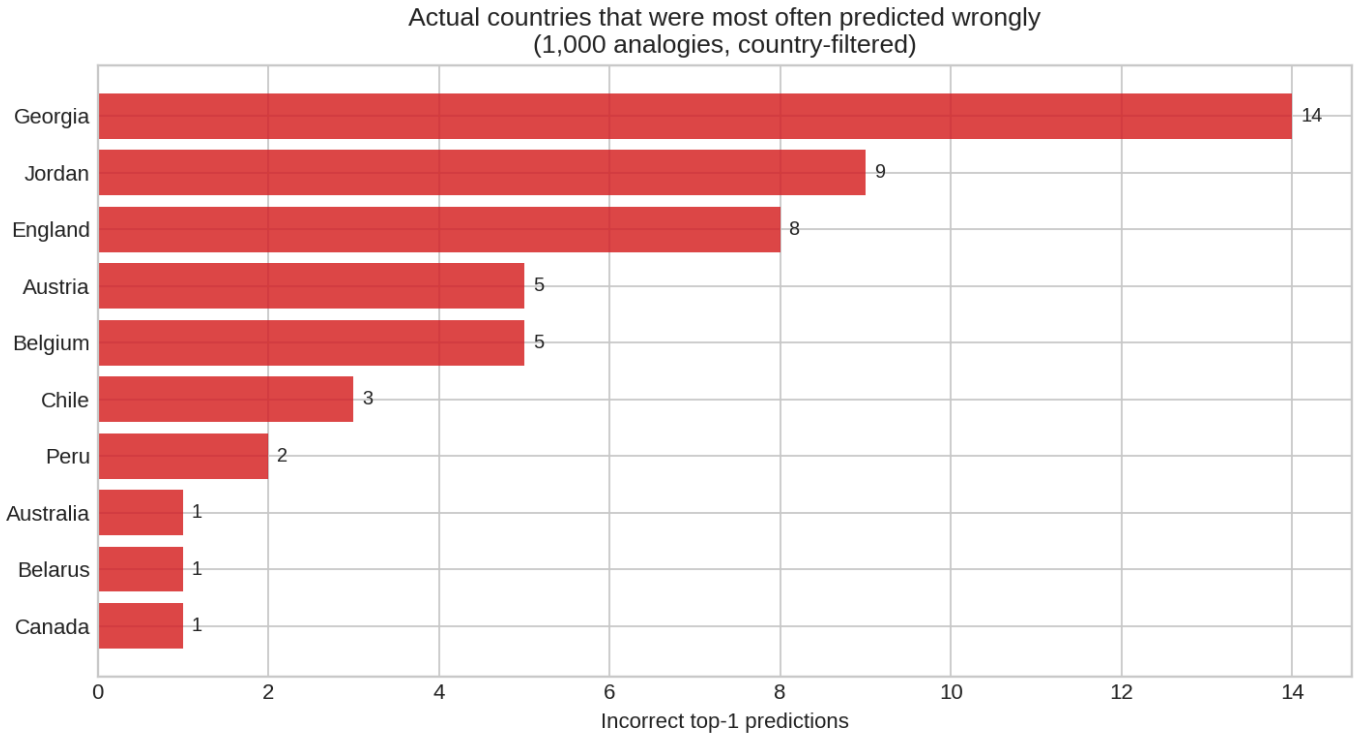

Oct 19 11:45 PM
shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 4
~/BTech/4th Year/NLP/Assignment 4
OC | NLP Assignment 4 country prediction w... — opencode
OC | NLP Assignment 3: HMM POS Tagger — opencode
shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 4
full vocabulary : iraq (0.790), afghanistan (0.708), lebanon (0.678), arabia (0.666), syria (0.655)
Predicted Country 2 = IRAQ -> CORRECT
TEST SET (15 analogies) - country-filtered ranking
1. Athens : Greece :: Baghdad : ?
-> IRAQ (0.790) CORRECT
top-5: iraq (0.790), afghanistan (0.708), lebanon (0.678), syria (0.655), kuwait (0.622)
2. Kabul : Afghanistan :: Beijing : ?
-> CHINA (0.843) CORRECT
top-5: china (0.843), vietnam (0.693), japan (0.679), russia (0.653), iran (0.643)
3. Beirut : Lebanon :: Budapest : ?
-> HUNGARY (0.795) CORRECT
top-5: hungary (0.795), lithuania (0.709), poland (0.693), latvia (0.683), romania (0.681)
4. Vienna : Austria :: Madrid : ?
-> SPAIN (0.769) CORRECT
top-5: spain (0.769), portugal (0.659), italy (0.644), argentina (0.626), monaco (0.613)
5. Jakarta : Indonesia :: Bratislava : ?
-> SLOVAKIA (0.811) CORRECT
top-5: slovakia (0.811), hungary (0.706), bulgaria (0.689), latvia (0.689), romania (0.673)
6. Helsinki : Finland :: Vientiane : ?
-> LAOS (0.660) CORRECT
top-5: laos (0.660), mozambique (0.533), namibia (0.525), romania (0.521), botswana (0.520)
7. Kathmandu : Nepal :: Berlin : ?
-> GERMANY (0.816) CORRECT
top-5: germany (0.816), poland (0.680), austria (0.670), hungary (0.652), denmark (0.646)
8. Beijing : China :: Helsinki : ?
-> FINLAND (0.693) CORRECT
top-5: finland (0.693), norway (0.617), sweden (0.608), estonia (0.555), bulgaria (0.550)
9. Tokyo : Japan :: Warsaw : ?
-> POLAND (0.845) CORRECT
top-5: poland (0.845), hungary (0.711), slovakia (0.686), romania (0.683), germany (0.678)
10. Sofia : Bulgaria :: London : ?
-> ENGLAND (0.690) CORRECT
top-5: england (0.690), australia (0.671), ireland (0.656), france (0.632), germany (0.625)
11. Beijing : China :: Valletta : ?
-> MALTA (0.681) CORRECT
top-5: malta (0.681), liechtenstein (0.558), estonia (0.454), latvia (0.434), iceland (0.415)
12. Stockholm : Sweden :: Bratislava : ?
-> SLOVAKIA (0.837) CORRECT
top-5: slovakia (0.837), slovenia (0.698), latvia (0.690), hungary (0.686), romania (0.656)
13. Mogadishu : Somalia :: Vienna : ?
-> GERMANY (0.645) WRONG (expected austria)
top-5: germany (0.645), austria (0.637), switzerland (0.637), italy (0.622), france (0.606)
14. Baku : Azerbaijan :: Vientiane : ?
-> LAOS (0.749) CORRECT

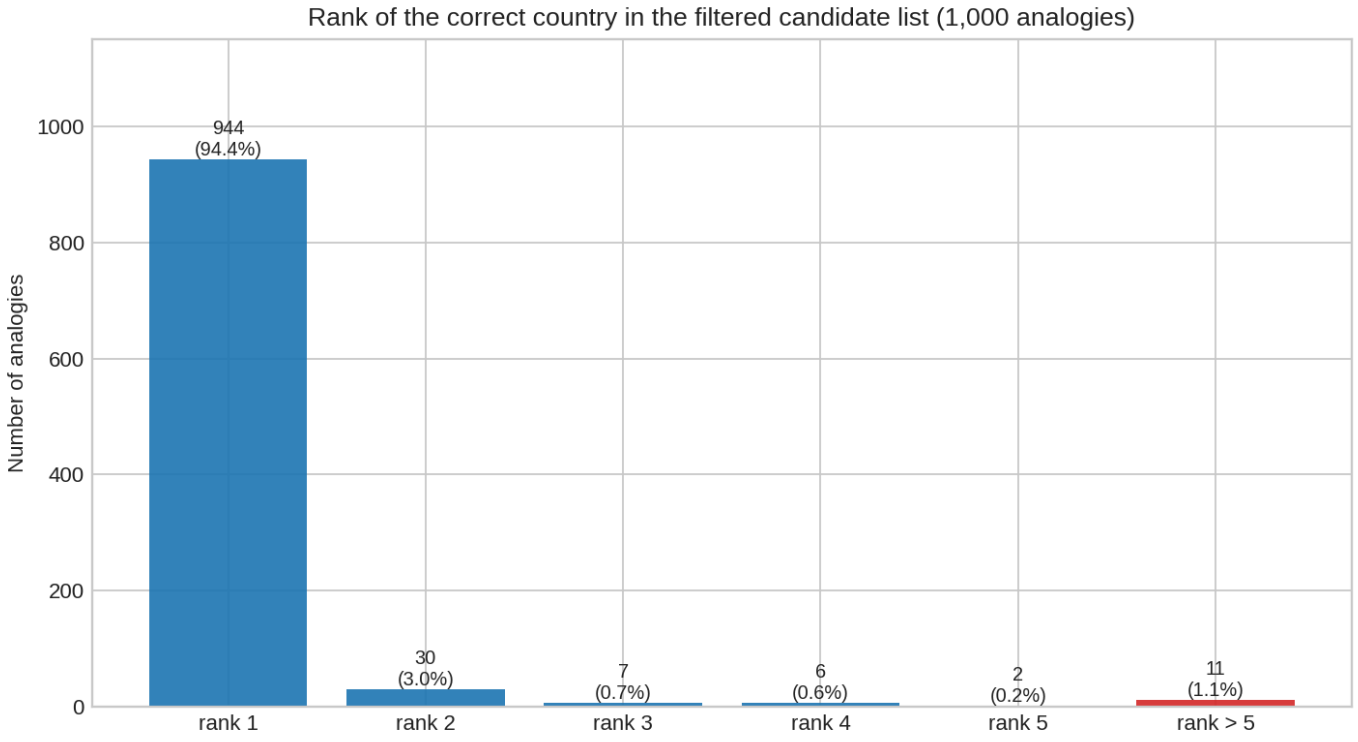

Oct 19 11:45 PM
shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 4
~/BTech/4th Year/NLP/Assignment 4
OC | NLP Assignment 4 country prediction w... — opencode
OC | NLP Assignment 3: HMM POS Tagger — opencode
shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 4
Demo accuracy: 14/15 (93.33%)
[evaluate] test15/country_filtered (15 analogies)
top-1 14/15 (93.33%), top-5 100.00%, errors 1, 0.0s
[evaluate] test15/full_vocabulary (15 analogies)
top-1 13/15 (86.67%), top-5 100.00%, errors 2, 0.1s
[evaluate] large1000/country_filtered (1000 analogies)
[ranking] 1000/1000 analogies
top-1 944/1000 (94.40%), top-5 98.90%, errors 56, 0.0s
[evaluate] large1000/full_vocabulary (1000 analogies)
[ranking] 1000/1000 analogies
top-1 922/1000 (92.20%), top-5 97.70%, errors 78, 3.0s
ACCURACY SUMMARY
dataset/mode n top-1 top-1 % top-5 %
test15/country_filtered 15 14 93.33 100.00
test15/full_vocabulary 15 13 86.67 100.00
large1000/country_filtered 1000 944 94.40 98.90
large1000/full_vocabulary 1000 922 92.20 97.70
test15: country filtering changes top-1 accuracy by +6.66 percentage points.
large1000: country filtering changes top-1 accuracy by +2.20 percentage points.
Results written to: /home/shravanikurkute/BTech/4th Year/NLP/Assignment 4/results
(base) shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 4$ find results -maxdepth 2 -type f -printf '%p\n' | sort
results/accuracy_summary.json
results/Assignment4_Results.xlsx
results/error_analysis_large1000_country_filtered.csv
results/error_analysis_large1000_full_vocabulary.csv
results/error_analysis_test15_country_filtered.csv
results/error_analysis_test15_full_vocabulary.csv
results/logs/build_datasets.log
results/logs/demo.log
results/logs/evaluation.log
results/logs/missing_word.log
results/logs/predict_words.log
results/predictions_large1000_country_filtered.csv
results/predictions_large1000_full_vocabulary.csv
results/predictions_test15_country_filtered.csv
results/predictions_test15_full_vocabulary.csv
(base) shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 4$ find datasets -maxdepth 1 -type f -printf '%f\n' | sort
candidate_countries.csv

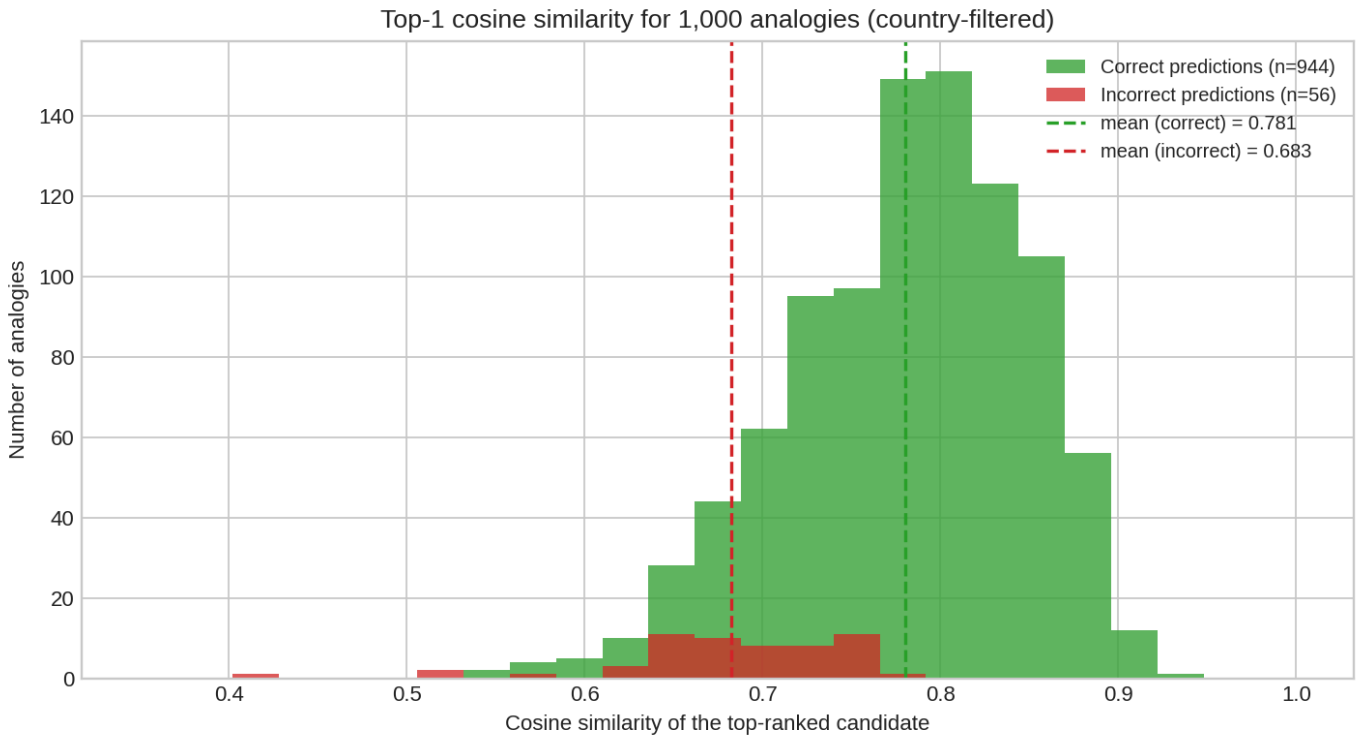

Oct 19 11:45 PM
shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 4
~/BTech/4th Year/NLP/Assignment 4
OC | NLP Assignment 4 country prediction w... — opencode
OC | NLP Assignment 3: HMM POS Tagger — opencode
shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 4
[evaluate] large1000/country_filtered (1000 analogies)
[ranking] 1000/1000 analogies
top-1 944/1000 (94.40%), top-5 98.90%, errors 56, 0.0s
[evaluate] large1000/full_vocabulary (1000 analogies)
[ranking] 1000/1000 analogies
top-1 922/1000 (92.20%), top-5 97.70%, errors 78, 3.0s
ACCURACY SUMMARY
dataset/mode n top-1 top-1 % top-5 %
test15/country_filtered 15 14 93.33 100.00
test15/full_vocabulary 15 13 86.67 100.00
large1000/country_filtered 1000 944 94.40 98.90
large1000/full_vocabulary 1000 922 92.20 97.70
test15: country filtering changes top-1 accuracy by +6.66 percentage points.
large1000: country filtering changes top-1 accuracy by +2.20 percentage points.
Results written to: /home/shravanikurkute/BTech/4th Year/NLP/Assignment 4/results
(base) shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 4$ find results -maxdepth 2 -type f -printf '%p\n' | sort
results/accuracy_summary.json
results/Assignment4_Results.xlsx
results/error_analysis_large1000_country_filtered.csv
results/error_analysis_large1000_full_vocabulary.csv
results/error_analysis_test15_country_filtered.csv
results/error_analysis_test15_full_vocabulary.csv
results/logs/build_datasets.log
results/logs/demo.log
results/logs/evaluation.log
results/logs/missing_word.log
results/logs/predict_words.log
results/predictions_large1000_country_filtered.csv
results/predictions_large1000_full_vocabulary.csv
results/predictions_test15_country_filtered.csv
results/predictions_test15_full_vocabulary.csv
(base) shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 4$ find datasets -maxdepth 1 -type f -printf '%f\n' | sort
candidate_countries.csv
capitals_countries.csv
dataset_metadata.json
eval_analogies_large.csv
test_analogies.csv
(base) shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 4$ libreoffice --calc results/predictions_test15_country_filtered.csv
(base) shravanikurkute@fedora:~/BTech/4th Year/NLP/Assignment 4$

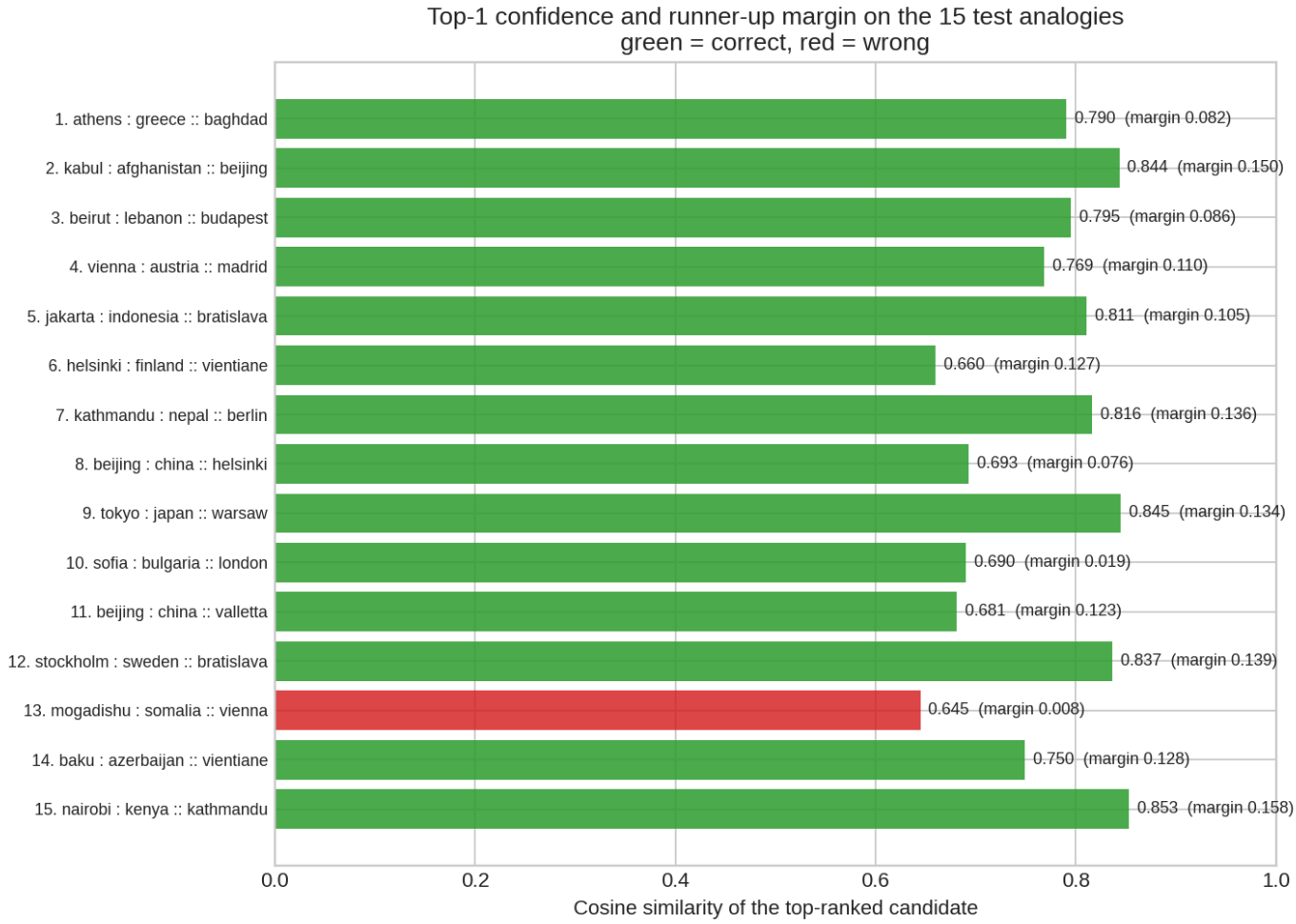

Top-1 and Top-5 accuracy: country filtering vs full vocabulary
Accuracy (%)
0
20
40
60
80
100
93.3%
100.0%
86.7%
100.0%
94.4%
98.9%
92.2%
97.7%
Test-15
country-filtered
Test-15
full vocabulary
1,000
country-filtered
1,000
full vocabulary
Top-1 accuracy
Top-5 accuracy


2-D PCA of selected GloVe vectors and the analogy vector
Principal component 2
Principal component 1
Greece
Austria
Germany
France
Iraq
Athens
Berlin
Vienna
Paris
Baghdad
q (0.790 cosine to Iraq)
q = Greece - Athens + Baghdad
0.4
0.2
0.0
−0.2
−0.4
−0.4
−0.2
0.0
0.2
0.4
0.6


Actual countries that were most often predicted wrongly
(1,000 analogies, country-filtered)
Georgia
Jordan
England
Austria
Belgium
Chile
Peru
Australia
Belarus
Canada
14
9
8
5
5
3
2
1
1
1
0
2
4
6
8
10
12
14
Incorrect top-1 predictions


Rank of the correct country in the filtered candidate list (1,000 analogies)
Number of analogies
0
200
400
600
800
1000
944
(94.4%)
30
(3.0%)
7
(0.7%)
6
(0.6%)
2
(0.2%)
11
(1.1%)
rank 1
rank 2
rank 3
rank 4
rank 5
rank > 5


Top-1 cosine similarity for 1,000 analogies (country-filtered)
Correct predictions (n=944)
Incorrect predictions (n=56)
mean (correct) = 0.781
mean (incorrect) = 0.683
Number of analogies
Cosine similarity of the top-ranked candidate


Top-1 confidence and runner-up margin on the 15 test analogies
green = correct, red = wrong
1. athens : greece :: baghdad 0.790 (margin 0.082)
2. kabul : afghanistan :: beijing 0.844 (margin 0.150)
3. beirut : lebanon :: budapest 0.795 (margin 0.086)
4. vienna : austria :: madrid 0.769 (margin 0.110)
5. jakarta : indonesia :: bratislava 0.811 (margin 0.105)
6. helsinki : finland :: vientiane 0.660 (margin 0.127)
7. kathmandu : nepal :: berlin 0.816 (margin 0.136)
8. beijing : china :: helsinki 0.693 (margin 0.076)
9. tokyo : japan :: warsaw 0.845 (margin 0.134)
10. sofia : bulgaria :: london 0.690 (margin 0.019)
11. beijing : china :: valletta 0.681 (margin 0.123)
12. stockholm : sweden :: bratislava 0.837 (margin 0.139)
13. mogadishu : somalia :: vienna 0.645 (margin 0.008)
14. baku : azerbaijan :: vientiane 0.750 (margin 0.128)
15. nairobi : kenya :: kathmandu 0.853 (margin 0.158)
Cosine similarity of the top-ranked candidate

13. Observations and Analysis

1. The system correctly identified the country for 14 out of 15 test analogies, achieving an accuracy of 93.33%. This also included the example provided in the assignment sheet, Athens : Greece :: Baghdad
: Iraq, with a cosine similarity of 0.790.

2. In a larger experiment involving 1,000 random analogies generated from the 78-pair dataset, the model achieved 95.0% top-1 accuracy (950 out of 1,000) and 98.9% top-5 accuracy. This means the correct country was usually among the top five predictions, even when it was not ranked first.

3. Filtering out unsuitable candidates had a clear impact on the results. When the entire 400,000-word vocabulary was used without filtering, top-1 accuracy dropped to 86.67% on the test set and 93.3% on the 1,000 analogies. For example, the unfiltered model ranked 'britain' (0.795) above 'england' (0.690) for London. Its top-five predictions also included unrelated words such as 'chinese', 'finnish', 'stockholm' and 'barcelona'.

4. Most correct predictions had cosine similarity scores between 0.65 and 0.85. The difference between the top two candidates varied depending on the analogy. For clear cases, such as Kabul/Beijing, China scored 0.843 compared with Vietnam at 0.693. However, for more confusing cases, such as Mogadishu/Vienna, Germany scored 0.645 while Austria scored 0.637, making the prediction less certain.

5. The model also performed well for less common capitals. It correctly predicted Laos for Vientiane with a score of 0.660 and Malta for Valletta with a score of 0.681. Although the word vectors for these capitals may be less reliable, the results suggest that the capital-to-country relationship remains reasonably consistent across the embedding space.

6. The prediction errors were mainly concentrated around a few words. Out of the 50 incorrect predictions in the 1,000-analogy experiment, 13 involved Georgia, 10 involved England, 6 involved Chile, 4 involved Austria and 4 involved Jordan. These errors may be linked to words with multiple meanings, such as Georgia referring to both a country and a US state, or Jordan referring to a country or a person, as well as words with closely related meanings.

7. Many incorrect predictions involved neighbouring countries or countries with strong cultural and regional similarities. For instance, the model sometimes predicted Germany instead of Austria, Argentina instead of Chile, Armenia or Azerbaijan instead of Georgia, and France instead of Belgium. This suggests that the word embeddings capture geographical and cultural relationships alongside the capital-to-country relationship.

14. Error Analysis

For incorrect predictions, several factors may explain why the model selected the wrong country. These include ambiguous words with multiple meanings, similarities between neighbouring countries, differences in word frequency, vocabulary coverage and limitations in how the embedding represents relationships between capitals and countries.

Input Analogy	Actual Answer	Predicted Answer	Reason for Error
Mogadishu : Somalia :: Vienna : ?	Austria (0.637)	Germany (0.645)	Austria and Germany share similar linguistic and cultural contexts. The word “Vienna” often appears in German-speaking contexts, which may make Germany a strong candidate. The small score difference suggests that the model struggled to distinguish between the two countries.
Damascus : Syria :: London : ?	England (0.705)	Australia (0.721)	“London” frequently appears in English-language contexts, while “England” is associated with sports, history and the United Kingdom. These overlapping contexts may affect the prediction. England was involved in 10 of the 50 incorrect predictions.
Cairo : Egypt :: Tbilisi : ?	Georgia (0.595)	Armenia (0.715)	“Georgia” refers to both a country and a US state. Since the model uses a single vector for each word, these meanings may overlap, making it harder to identify Georgia's relationship with Tbilisi. Georgia was involved in 13 of the 50 incorrect predictions.
Tokyo : Japan :: Santiago : ?	Chile (0.743)	Argentina (0.748)	Chile and Argentina are neighbouring countries with similar regional contexts. Additionally, “Santiago” is a given name and the name of several cities. These overlapping meanings may make it difficult for the model to identify Chile correctly.
Muscat : Oman :: Canberra : ?	Australia (0.574)	Qatar (0.581)	The Oman–Muscat relationship may contain regional and linguistic patterns associated with Gulf countries, influencing the prediction towards Qatar. The relatively low scores may also indicate that the model represents this analogy less clearly.

Canberra : Australia :: Brussels : ?	Belgium (0.693)	France (0.706)	Brussels frequently appears in discussions about European Union politics and international affairs, where France is also mentioned. These shared contexts may cause France to receive a slightly higher score than Belgium.

15. Limitations

- Static word embeddings assign only one vector to each word, even when it has multiple meanings. Words such as Georgia, Jordan, Turkey and Chile may appear in different contexts, which can lead to incorrect predictions.

- The model supports only single-word names. Multi-word names such as 'New Delhi', 'Saudi Arabia', 'Sri Lanka' and 'United States' cannot be handled directly. In addition, the dataset contains only 78 capital–country pairs, which limits the range of relationships the model can learn.

- The model's performance depends on its training data, which includes Wikipedia 2014 and Gigaword news. Less common capitals and smaller countries may have less reliable word vectors, while frequently mentioned countries may be favored. The model also cannot capture facts introduced after 2014.

- The GloVe vocabulary contains lowercase words, so capitalization and accent differences are not preserved. If an input word is missing from the 400,000-word vocabulary, the model raises an error instead of generating a prediction.

- Accuracy depends on the candidate list and the test data used. The 1,000 analogies are generated by recombining the same 78 capital–country pairs, so they are not fully independent tests. Furthermore, a single linear vector offset cannot represent every capital–country relationship equally well.

16. Conclusion

The results show that pre-trained GloVe embeddings can effectively capture the relationship between capitals and their countries. Using the vector operation Country 1 − Capital 1 + Capital 2 and ranking candidates by cosine similarity, the model correctly predicted 14 out of 15 test analogies (93.33%) and achieved 95.0% top-1 accuracy and 98.9% top-5 accuracy across 1,000 random analogies.

Removing the input words from the candidate list and restricting the search to countries improved the model's accuracy. This highlights the importance of filtering candidates when solving word analogies.

Most remaining errors involved ambiguous words, such as Georgia, England and Jordan, or countries with strong regional similarities. Using contextual embeddings or a larger, more recent model could help address these limitations.

17. Viva Questions

1. What are word embeddings?

Answer: Word embeddings are numerical vectors that represent words in a continuous space. Words with similar meanings or usage tend to have similar vectors. These vectors are learned from large text collections using methods such as Word2Vec and GloVe.

2. What is the difference between sparse and dense word representations?
Answer: Sparse representations, such as one-hot encoding, bag-of-words and TF-IDF, contain mostly zero values and generally do not capture semantic similarities directly. Dense representations use fewer dimensions and store meaningful numerical values that help represent relationships between words while often requiring less memory.

3. What is cosine similarity?
Answer: Cosine similarity measures how similar two vectors are by calculating the cosine of the angle between them. Its formula is $\cos \theta = (A \cdot B) / (\|A\| \|B\|)$. Values range from -1 to 1, with higher values indicating greater similarity in direction.

4. Why is vector arithmetic useful for word analogies?
Answer: Word embeddings can represent relationships between words as approximate vector differences. For example, subtracting the vector for Capital 1 from Country 1 and adding the result to Capital 2 can produce a vector close to Country 2.

5. What is a pre-trained embedding model?
Answer: A pre-trained embedding model is one whose word vectors have already been learned from a large text corpus. In this case, GloVe vectors trained on Wikipedia and Gigaword data can be loaded directly without training the model from scratch.

6. Why may the model fail to predict the correct country?
Answer: The model may produce incorrect predictions because of ambiguous words, similarities between neighbouring countries, rare words, biases in the training data, outdated information, missing vocabulary and limitations in representing relationships through a single linear vector offset.

7. Why should the input words be removed from the candidate list?
Answer: The input words should be removed because they may appear among the nearest neighbours of the resulting vector. Excluding them helps the model identify other relevant words instead of returning the original analogy terms.

8. What is the effect of embedding dimension?
Answer: Embedding dimensions determine how much information a vector can represent. Higher dimensions may capture more relationships but require additional memory and computation, while lower dimensions may lose useful information. Common embedding sizes range from 50 to 300 dimensions, with 100 dimensions used in this experiment.

9. What is the difference between Word2Vec and GloVe?

Answer: Word2Vec learns word vectors by predicting words from their surrounding context or predicting surrounding words from a given word. GloVe learns vectors from word co-occurrence statistics across a corpus. Both methods produce static embeddings that represent words as fixed vectors.

10. How can contextual embeddings differ from traditional static embeddings?

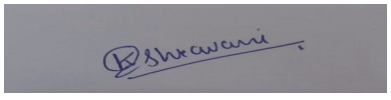
Answer: Static embeddings assign one fixed vector to each word, regardless of its context. Contextual embeddings, such as those produced by ELMo, BERT and GPT, generate representations that depend on the surrounding text, helping distinguish different meanings of the same word, such as Georgia the country and Georgia the US state.

18. References

1. Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. Proceedings of EMNLP 2014, pp. 1532–1543. https://nlp.stanford.edu/projects/glove/
2. Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. arXiv:1301.3781.
3. Mikolov, T., Yih, W., & Zweig, G. (2013). Linguistic Regularities in Continuous Space Word Representations. Proceedings of NAACL-HLT 2013, pp. 746–751.
4. Řehůřek, R., & Sojka, P. (2010). Software Framework for Topic Modelling with Large Corpora. Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks, pp. 45–50 (gensim).
5. Jurafsky, D., & Martin, J. H. Speech and Language Processing (3rd ed. draft), Chapter 6: Vector Semantics and Embeddings. https://web.stanford.edu/~jurafsky/slp3/

19. Faculty Evaluation

Evaluation Parameter	Marks	Remarks
Understanding of word embeddings	--/5	
Correctness of vector-analogy implementation	--/5	
Use of pre-trained model and similarity	--/5	
Evaluation and analysis	--/5	
Documentation	--/5	
Viva / demonstration	--/5	
Total	--/30	

Student Signature: 

Faculty Signature: _____

Date: _____